



รายงานโครงการคอมพิวเตอร์
เรื่อง โปรแกรมคำนวณเซตบิตประสิทธิภาพสูง (Set Bitwise Calculator)

โดย

XXXXXXXXXXXXXx

XXXXXXx

XXXXXXXXXXXXXX

เสนอ

YYYYYYYYYYYY

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี
รายงานฉบับนี้เป็นส่วนประกอบของโครงการในรายวิชา
CPE100 | COMPUTER PROGRAMMING FOR ENGINEERS
ภาคการศึกษา: 1/2568



บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

ในปัจจุบัน ทฤษฎีเซต (Set Theory) ถือเป็นรากฐานสำคัญทางคณิตศาสตร์ที่ถูกนำมาประยุกต์ใช้อย่างแพร่หลายในวิทยาการคอมพิวเตอร์ ทั้งในด้านฐานข้อมูล (Database Systems), ตรรกศาสตร์คอมพิวเตอร์, และการออกแบบอัลกอริทึม อย่างไรก็ตาม การจัดการและจัดเก็บข้อมูลเซตในรูปแบบดั้งเดิมโดยใช้อาร์เรย์ (Array) หรือรายการโยง (Linked List) มักประสบปัญหาด้านประสิทธิภาพเมื่อปริมาณข้อมูลมีขนาดใหญ่ โดยเฉพาะความซับซ้อนเชิงเวลา (Time Complexity) ในการดำเนินการพื้นฐาน เช่น การหาอินเตอร์เซกชัน (Intersection) หรือผลต่าง (Difference) ที่จำเป็นต้องมีการเปรียบเทียบสมาชิกทุกตัว ซึ่งส่งผลให้การประมวลผลมีความล่าช้าและใช้ทรัพยากรหน่วยความจำอย่างสิ้นเปลือง

จากปัญหาดังกล่าว คณะผู้จัดทำจึงเล็งเห็นถึงความสำคัญในการนำเทคนิคการดำเนินการระดับบิต (Bitwise Operations) หรือการแทนที่ข้อมูลด้วยบิตเวกเตอร์ (Bit Vector Representation) มาประยุกต์ใช้ในการแก้ปัญหา การแทนสมาชิกของเซตด้วยสถานะของบิต (0 และ 1) ช่วยให้การดำเนินการทางตรรกศาสตร์ระหว่างเซตสามารถทำได้พร้อมกันที่หลายสมาชิกภายในรอบสัญญาณนาฬิกาเดียว (Parallel execution at bit level) ซึ่งช่วยลดความซับซ้อนของเวลาลงได้อย่างมีนัยสำคัญ และยังช่วยลดพื้นที่ในการจัดเก็บข้อมูลลงเหลือเพียง 1 บิตต่อ 1 สมาชิก ทำให้ระบบมีความรวดเร็วและประหยัดทรัพยากรมากกว่าวิธีดั้งเดิมหลายเท่าตัว

ดังนั้น โครงการงาน "การจัดการและวิเคราะห์เซตด้วยการแทนค่าแบบบิตเวกเตอร์ (Bit-Mapped Set Analyzer)" จึงถูกพัฒนาขึ้นเพื่อสาธิตประสิทธิภาพของอัลกอริทึมดังกล่าวผ่านโปรแกรมคอมพิวเตอร์ภาษา C ที่ทำงานบนระบบ Console โดยมุ่งเน้นการพัฒนาระบบคำนวณเซตที่ครบวงจร รองรับการดำเนินการพื้นฐาน ($\cup, \cap, -, \sim$) ควบคู่ไปกับการวิเคราะห์ค่าสถิติพื้นฐาน (ค่าเฉลี่ยและมัธยฐาน) รวมถึงระบบจัดการไฟล์เพื่อการนำเข้าและส่งออกข้อมูล ผลลัพธ์ที่ได้จากโครงการนี้จะป็นต้นแบบของซอฟต์แวร์ที่มีประสิทธิภาพสูง ซึ่งแสดงให้เห็นถึงความสำคัญของการเลือกใช้โครงสร้างข้อมูลที่เหมาะสมในการพัฒนาซอฟต์แวร์ระดับระบบ

1.2 วัตถุประสงค์

1.2.1] เพื่อพัฒนาโปรแกรมคอมพิวเตอร์สำหรับการคำนวณและจัดการเซตทางคณิตศาสตร์ที่มีประสิทธิภาพสูง โดยประยุกต์ใช้เทคนิค การดำเนินการระดับบิต (Bitwise Operations) ในการลดความซับซ้อนเชิงเวลาและพื้นที่หน่วยความจำ

1.2.2] เพื่อศึกษาและประยุกต์ใช้อัลกอริทึมในการดำเนินการพื้นฐานทางเซต ได้แก่ ยูเนียน (Union), อินเตอร์เซกชัน (Intersection), ผลต่าง (Difference), ผลต่างสมมาตร (Symmetric Difference) และคอมพลีเมนต์ (Complement) ผ่านโครงสร้างข้อมูลแบบบิตเวกเตอร์



1.2.3] เพื่อเพิ่มขีดความสามารถในการวิเคราะห์ข้อมูลเชิงลึกให้กับโปรแกรม โดยการพัฒนาฟังก์ชันสำหรับคำนวณค่าสถิติเชิงพรรณนา ได้แก่ ค่าเฉลี่ยเลขคณิต (Mean) และ ค่ามัธยฐาน (Median) ของสมาชิกภายในเซต

1.2.4] เพื่อพัฒนาระบบจัดการข้อมูลที่มีความยืดหยุ่น รองรับการนำเข้าข้อมูลเซต (Import) และการบันทึกผลลัพธ์การคำนวณ (Export) ผ่านไฟล์ข้อความ (Text File) เพื่อความสะดวกในการใช้งานจริง

1.2.5] เพื่อสร้างซอฟต์แวร์ที่มีโครงสร้างแบบโมดูลาร์ (Modular Design) และมีส่วนติดต่อประสานงานกับผู้ใช้ (User Interface) บนระบบ Console ที่เป็นระเบียบและใช้งานง่าย

1.3 ประโยชน์ที่คาดว่าจะได้รับ

1.3.1] ได้ซอฟต์แวร์ต้นแบบ (Prototype Software) สำหรับการคำนวณและวิเคราะห์เซตที่มีประสิทธิภาพสูง มีความรวดเร็วในการประมวลผลและใช้ทรัพยากรหน่วยความจำต่ำ ซึ่งสามารถนำไปต่อยอดใช้งานจริงหรือใช้เป็นสื่อการเรียนการสอนในรายวิชาที่เกี่ยวข้องได้

1.3.2] ได้รับความรู้ความเข้าใจเชิงลึกเกี่ยวกับหลักการทำงานของ การดำเนินการระดับบิต (Bitwise Operations) และสามารถประยุกต์ใช้ในการแก้ปัญหาทางคอมพิวเตอร์ที่ซับซ้อน เพื่อลดความซับซ้อนของอัลกอริทึม (Algorithm Optimization) แทนการใช้วิธีการวนลูปแบบดั้งเดิม

1.3.3] มีเครื่องมืออำนวยความสะดวกทางคณิตศาสตร์ที่สามารถดำเนินการทางเซตครบวงจร ($\cup$, $\cap$, $-$, $\sim$) พร้อมทั้งสามารถวิเคราะห์ค่าสถิติพื้นฐาน (ค่าเฉลี่ยและมัธยฐาน) ได้อย่างถูกต้องและแม่นยำ ช่วยลดความผิดพลาดจากการคำนวณด้วยตนเอง

1.3.4] ได้ทักษะในการพัฒนาซอฟต์แวร์เชิงระบบด้วยภาษา C ทั้งในด้านการจัดการหน่วยความจำ (Memory Management), การจัดการไฟล์ (File I/O) สำหรับนำเข้าและส่งออกข้อมูล, และการออกแบบโครงสร้างโปรแกรมแบบแยกส่วน (Modular Programming) ซึ่งเป็นพื้นฐานสำคัญของการเป็นนักพัฒนาซอฟต์แวร์มืออาชีพ

1.3.5] เป็นแนวทางในการออกแบบส่วนติดต่อผู้ใช้งาน (User Interface) บนระบบปฏิบัติการแบบ Console ที่เน้นความเรียบง่ายแต่ใช้งานได้จริง (Usability) และมีการจัดลำดับขั้นตอนการทำงาน (Workflow) ที่เป็นระบบ



1.3 สมาชิกภายในกลุ่มและการมอบหมายหน้าที่

ตรงนี้ใส่ไฟล์ด้วย โยงจาก setcalculator.c แล้วแยกแต่ละฟังก์ชันอีกที ให้แต่ละคน



บทที่ 2

วิธีการดำเนินงาน

2.1 ความสามารถของโปรแกรม

โปรแกรม Set Bitwise Calculator ถูกออกแบบมาให้มีความสามารถครอบคลุมกระบวนการจัดการเซต ตั้งแต่การนำเข้าข้อมูล การประมวลผล ไปจนถึงการจัดเก็บผลลัพธ์ โดยมีความสามารถหลักแบ่งออกเป็น 4 ส่วนดังนี้

2.1.1] การนำเข้าข้อมูลเซต (Set Data Import)

โปรแกรมรองรับการนำเข้าข้อมูลเซต (Input) เพื่อนำไปจัดเก็บในหน่วยความจำชั่วคราว (RAM) โดยใช้โครงสร้างข้อมูลแบบบิตเวกเตอร์ ซึ่งสามารถดำเนินการได้ 2 รูปแบบ คือ

2.1.1.1) การป้อนข้อมูลผ่านแป้นพิมพ์ (Keyboard Input): ผู้ใช้งานสามารถป้อนนิยามเซตที่ละหนึ่งเซตในรูปแบบมาตรฐานทางคณิตศาสตร์ เช่น $A = \{1, 2, 3\}$ โดยโปรแกรมจะมีระบบตรวจสอบความถูกต้องของรูปแบบข้อมูล (Syntax Validation) ก่อนบันทึก

```
=====
Input Set from Keyboard
=====
Format: Name = {1, 2, 3}
Enter set > SET1 = {1,2,3,4,5,6,7,8}
[+] New set stored: SET1

[Press Enter to continue...]
```

ภาพที่ 2.1 การป้อนข้อมูลผ่านแป้นพิมพ์ (Keyboard Input)

2.1.1.2) การนำเข้าข้อมูลจากไฟล์ (File Import): รองรับการอ่านไฟล์ข้อความ (.txt) ที่มีข้อมูลเซตจำนวนมาก โดยโปรแกรมจะทำการอ่านและแปลงข้อมูลทุกบรรทัดให้เป็นเซตในหน่วยความจำพร้อมกันครั้งเดียว ช่วยลดความซ้ำซ้อนในการป้อนข้อมูล

```
=====
Import from File
=====
Filename > sets.txt
Reading from 'sets.txt'...
[*] Set updated: A
[*] Set updated: B
[*] Set updated: C
[*] Set updated: D
[*] Set updated: E
[Done] Import finished.

[Press Enter to continue...]
```

```
sets.txt
1  A = {1,2,4,7}
2  B = {0,1,2,3,5}
3  C = {2,3,4,5,6}
4  D = {1,2,4,7}
5  E = {1,2,4,7}
```

ภาพที่ 2.2 การป้อนข้อมูลผ่านไฟล์ (File Import) และตัวอย่างไฟล์ sets.txt



2.1.2] การแสดงผลและการวิเคราะห์เชิงสถิติ (Display & Statistical Analysis)

โปรแกรมมีความสามารถในการแสดงรายละเอียดของเซตทั้งหมดที่มีอยู่ในระบบ โดยจะแสดงชื่อเซตและสมาชิกภายในเซต นอกจากนี้ ยังมีการประมวลผลค่าทางสถิติเบื้องต้นแบบเรียลไทม์ (Real-time Calculation) ให้กับทุกเซต ได้แก่

2.1.2.1) ค่าเฉลี่ยเลขคณิต (Mean): คำนวณจากผลรวมของสมาชิกหารด้วยจำนวนสมาชิก

2.1.2.2) ค่ามัธยฐาน (Median): คำนวณจากค่ากึ่งกลางของสมาชิกในเซต

```
=====
Summary of All Sets (with Stats)
=====
SET1      = { 1, 2, 3, 4, 5, 6, 7, 8 } [Mean: 4.50, Median: 4.50]
A         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
B         = { 0, 1, 2, 3, 5 } [Mean: 2.20, Median: 2.00]
C         = { 2, 3, 4, 5, 6 } [Mean: 4.00, Median: 4.00]
D         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
E         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
=====
[Press Enter to continue...]
```

ภาพที่ 2.3 หน้าจอแสดงผลสรุปเซตพร้อมค่าเฉลี่ยและค่ามัธยฐาน

2.1.3] การคำนวณดำเนินการทางเซต (Set Operations)

ส่วนประมวลผลหลักของโปรแกรมใช้เทคนิค Bitwise Logic ในการดำเนินการระหว่างเซต 2 เซต (Binary Operations) หรือ 1 เซต (Unary Operation) โดยให้ผลลัพธ์ที่แม่นยำและรวดเร็ว รองรับตัวดำเนินการดังนี้:

2.1.3.1) อินเตอร์เซกชัน (Intersection, $\cap$): (Bitwise AND)

2.1.3.2) ยูเนียน (Union, $\cup$): (Bitwise OR)

2.1.3.3) ผลต่างสมมาตร (Symmetric Difference, $\oplus$): (Bitwise XOR)

2.1.3.4) ผลต่าง (Difference, $-$): (Bitwise AND with NOT)

2.1.3.5) คอมพลีเมนต์ (Complement, $'$): (Bitwise NOT)

```
=====
Summary of All Sets (with Stats)
=====
SET1      = { 1, 2, 3, 4, 5, 6, 7, 8 } [Mean: 4.50, Median: 4.50]
A         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
B         = { 0, 1, 2, 3, 5 } [Mean: 2.20, Median: 2.00]
C         = { 2, 3, 4, 5, 6 } [Mean: 4.00, Median: 4.00]
D         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
E         = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
=====

[Calculation Mode]
1st Set Name > A
Operation (&, |, ^, -, ~) > &
2nd Set Name > B

Result: A & B
=====
RESULT      = { 1, 2 } [Mean: 1.50, Median: 1.50]
=====
Save this result? (y/n) > |
```

ภาพที่ 2.4 ตัวอย่างการคำนวณ Intersection และผลลัพธ์ที่ได้



2.1.4] การบันทึกและส่งออกข้อมูล (Data Export)

เพื่อความสะดวกในการนำผลลัพธ์ไปใช้งานต่อ โปรแกรมมีฟังก์ชันในการบันทึกสถานะปัจจุบันของเซตทั้งหมด รวมถึงเซตผลลัพธ์ที่เกิดจากการคำนวณ ลงสู่ไฟล์ข้อความ (.txt) ในรูปแบบที่มนุษย์อ่านเข้าใจได้ (Human-readable format) และสามารถนำไฟล์ดังกล่าวกลับมา Import เข้าสู่โปรแกรมได้อีกครั้งในภายหลัง

```
=====
Save to File
=====
Enter filename > New_Result_Set.txt
[Done] Results saved to 'New_Result_Set.txt'

[Press Enter to continue...]|
```

ภาพที่ 2.5 การบันทึกผลลัพธ์ลงไฟล์และการแจ้งเตือนเมื่อสำเร็จ



2.2 ภาพรวมการทำงานของระบบ (System Overview)

โปรแกรม Set Bitwise Calculator ถูกออกแบบมาให้มีความสามารถครอบคลุมกระบวนการจัดการเซต ตั้งแต่การนำเข้าข้อมูล การประมวลผล ไปจนถึงการจัดเก็บผลลัพธ์ โดยมีความสามารถหลักแบ่งออกเป็น 4 ส่วนดังนี้

2.2.1] ฟังก์ชัน main: เป็นฟังก์ชันหลักที่คอยวนลูปแสดงเมนู รับค่าจากผู้ใช้ และเรียกใช้ฟังก์ชันย่อยตามตัวเลือกที่เลือก

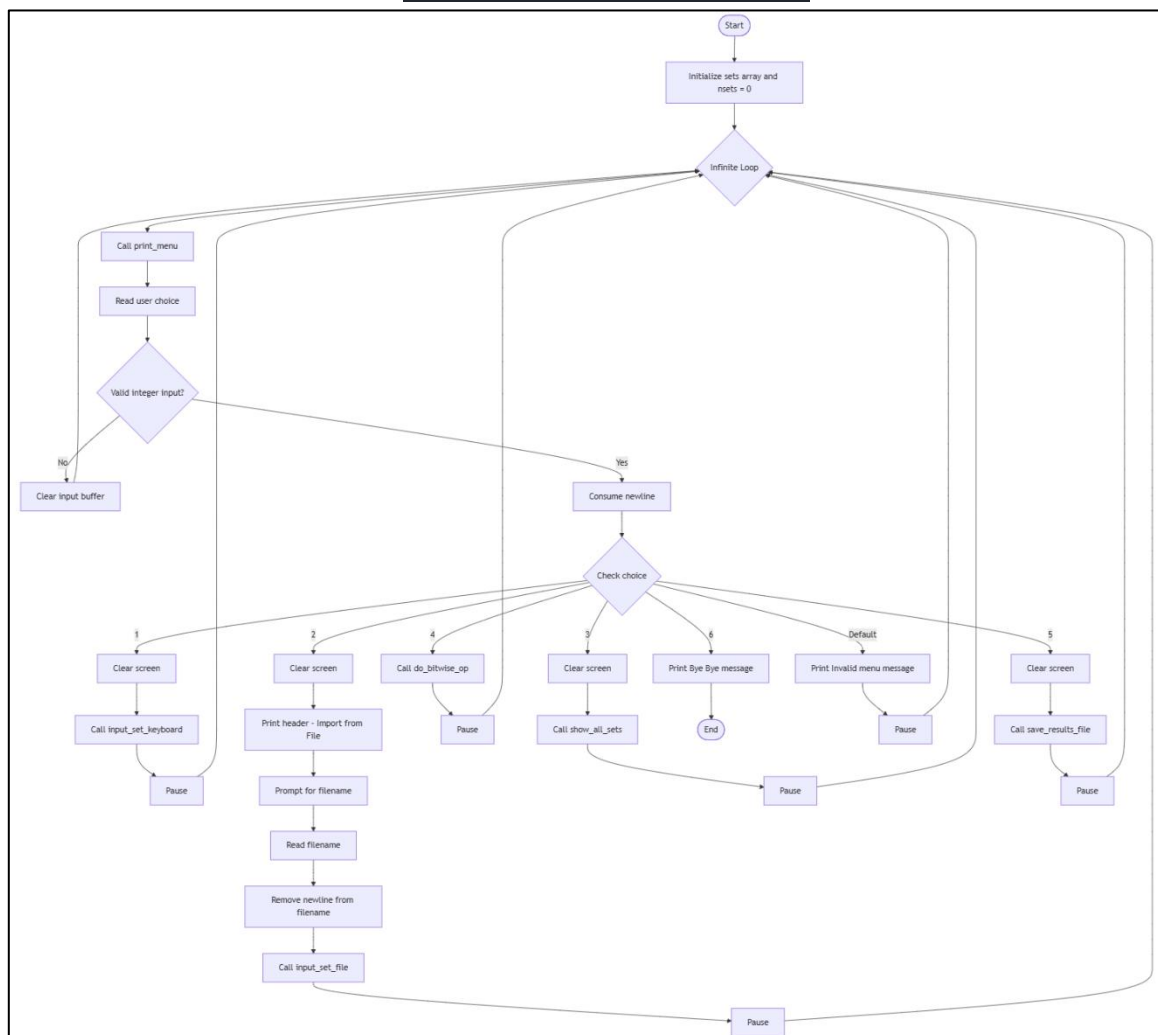
```
int main() {
    Set sets[MAX_SETS];
    int nsets = 0;

    while (1) {
        print_menu();
        int ch;
        char dummy[8];

        if (scanf("%d", &ch) != 1) {
            fgets(dummy, sizeof(dummy), stdin);
            continue;
        }
        getchar(); // Consume newline
    }
}
```

```
switch (ch) {
    case 1:
        ui_clear_screen();
        input_set_keyboard(sets, &nsets);
        ui_pause();
        break;
    case 2: {
        ui_clear_screen();
        ui_print_header("Import from File");
        char fname[128];
        printf("Filename > ");
        fgets(fname, sizeof(fname), stdin);
        fname[strlen(fname) - 1] = 0;
        input_set_file(fname, sets, &nsets);
        ui_pause();
        break;
    }
    case 3:
        ui_clear_screen();
        show_all_sets(sets, nsets);
        ui_pause();
        break;
}
```

```
case 4:
    do_bitwise_op(sets, &nsets);
    ui_pause();
    break;
case 5:
    ui_clear_screen();
    save_results_file(sets, nsets);
    ui_pause();
    break;
case 6:
    printf("\n Bye Bye! :)\n");
    return 0;
default:
    printf("\n [!] Invalid menu.\n");
    ui_pause();
}
```



ภาพที่ 2.6 โค้ดโปรแกรม และ Flowchart แสดงภาพรวมการทำงานของฟังก์ชัน main()



2.3 รายละเอียดฟังก์ชันและการทำงาน (Detailed Functions)

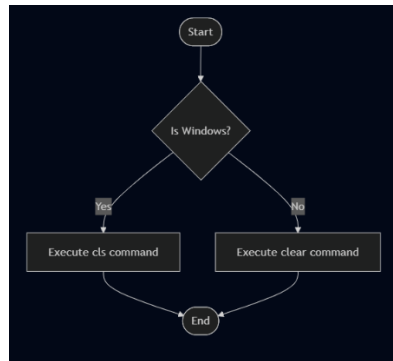
***หมายเหตุ* : หาก Code หรือ Flowchart ไม่ชัดเจน สามารถคลิกที่นี่ได้ (GitHub Repository)**

2.3.1 กลุ่มที่ 1 ส่วนติดต่อผู้ใช้งาน (UI Helpers):

2.3.1.1) ฟังก์ชัน ui_clear_screen()

```
set_calculator.c

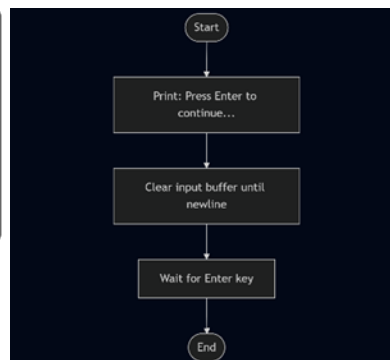
void ui_clear_screen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}
```



2.3.1.2) ฟังก์ชัน ui_pause()

```
set_calculator.c

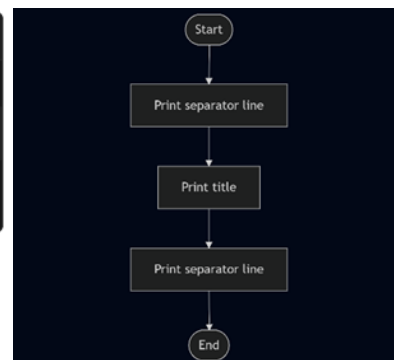
void ui_pause() {
    printf("\n[Press Enter to continue...]");
    char c;
    while ((c = getchar()) != '\n' && c != EOF);
    getchar();
}
```



2.3.1.3) ฟังก์ชัน ui_print_header()

```
set_calculator.c

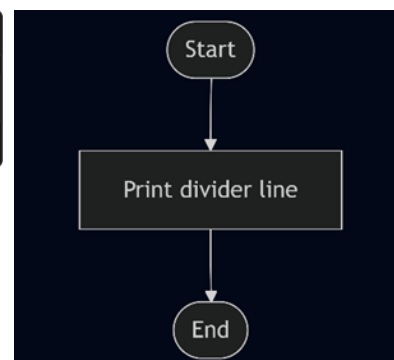
void ui_print_header(const char* title) {
    printf("\n=====\\n");
    printf(" %s\\n", title);
    printf("=====\\n");
}
```



2.3.1.4) ฟังก์ชัน ui_print_divider()

```
set_calculator.c

void ui_print_divider() {
    printf("-----\\n");
}
```





2.3.21 กลุ่มที่ 2 ฟังก์ชันจัดการบิต (Core Utility):

2.3.2.1) ฟังก์ชัน trim()

set_calculator.c

C++

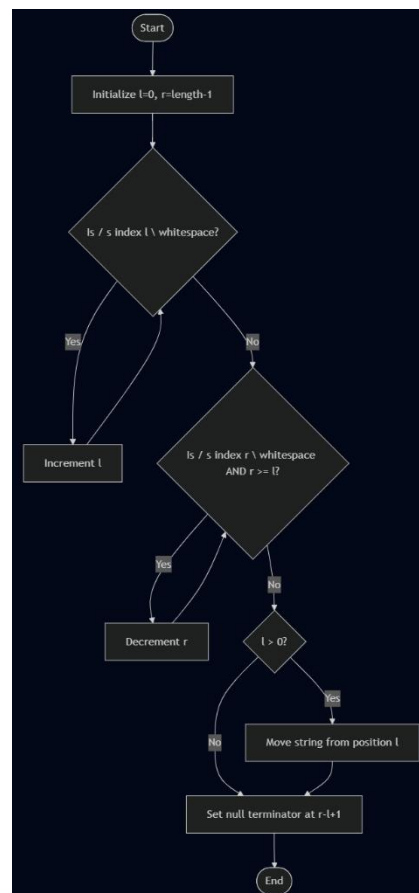
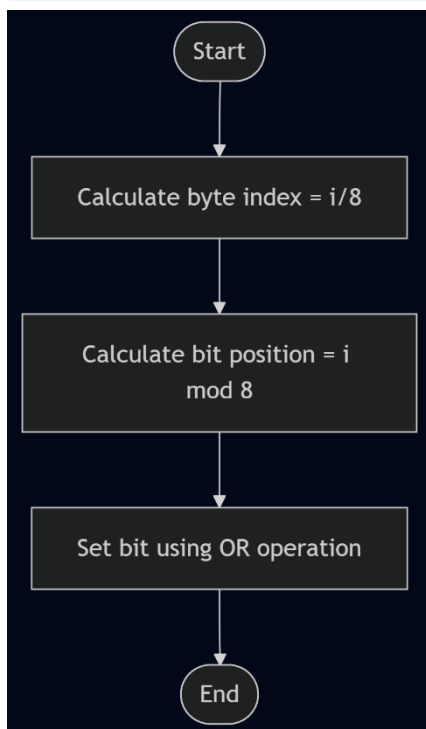
```
void trim(char* s) {  
    int l = 0, r = strlen(s) - 1;  
    while (isspace((unsigned char)s[l])) l++;  
    while (r >= l && isspace((unsigned char)s[r])) r--;  
    if (l > 0) memmove(s, s + l, r - l + 1);  
    s[r - l + 1] = 0;  
}
```

2.3.2.2) ฟังก์ชัน set_bit()

set_calculator.c

C++

```
void set_bit(unsigned char* bits, int i) {  
    bits[i/8] |= (1 << (i % 8));  
}
```

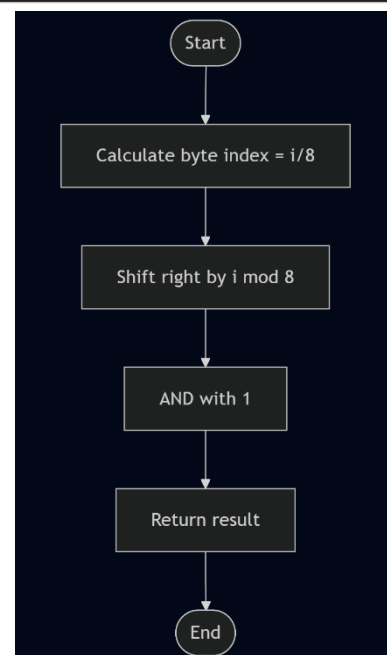


2.3.2.3) ฟังก์ชัน get_bit()

set_calculator.c

C++

```
int get_bit(const unsigned char* bits, int i) {  
    return (bits[i/8] >> (i % 8)) & 1;  
}
```

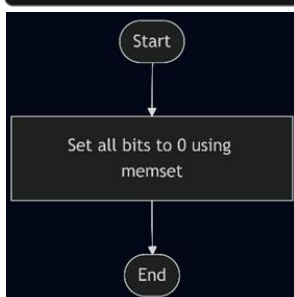


2.3.2.4) ฟังก์ชัน clear_bits()

set_calculator.c

C++

```
void clear_bits(unsigned char* bits) {  
    memset(bits, 0, BIT_ARR_LEN);  
}
```

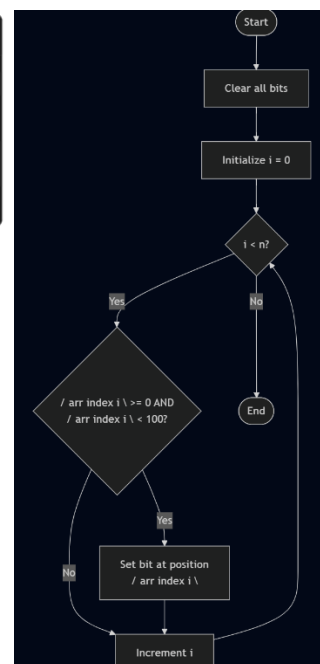




2.3.2.5) ฟังก์ชัน set_to_bits()

```
set_calculator.c SCSS

void set_to_bits(unsigned char* bits, int* arr, int n) {
    clear_bits(bits);
    for (int i = 0; i < n; ++i) {
        if (arr[i] >= 0 && arr[i] < UNIVERSAL_SET_SIZE) set_bit(bits, arr[i]);
    }
}
```



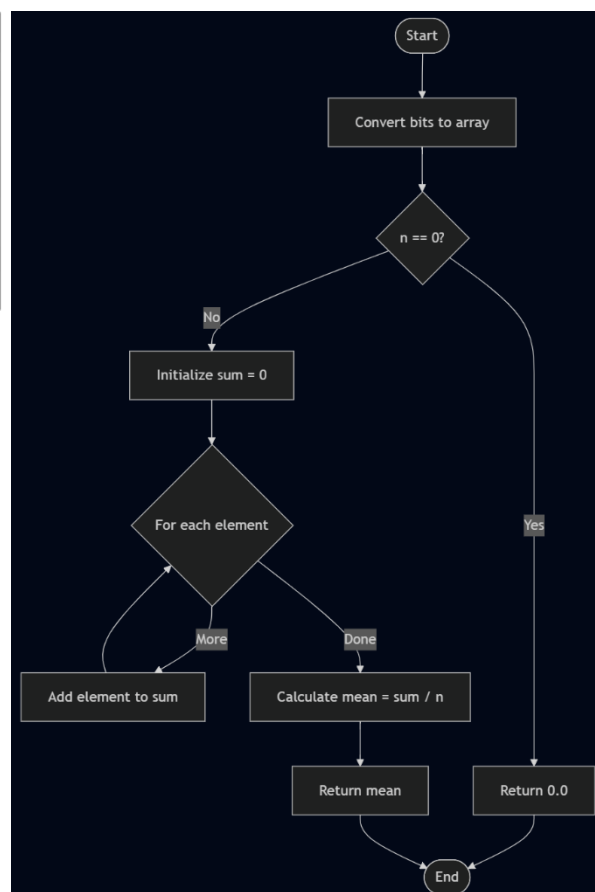
2.3.3] กลุ่มที่ 3 ฟังก์ชันสถิติ (Statistics):

2.3.3.1) ฟังก์ชัน calculate_mean()

```
set_calculator.c C++

double calculate_mean(const Set* s) {
    int arr[UNIVERSAL_SET_SIZE], n;
    bits_to_set(s->bits, arr, &n);
    if (n == 0) return 0.0;

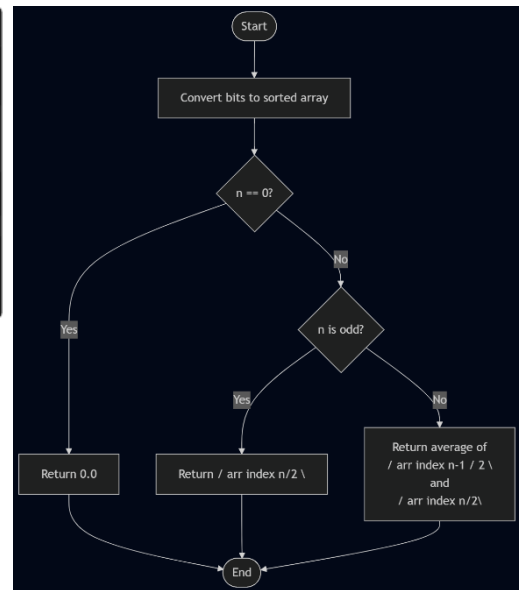
    double sum = 0;
    for (int i = 0; i < n; ++i) sum += arr[i];
    return sum / n;
}
```





2.3.3.2) ฟังก์ชัน calculate_median()

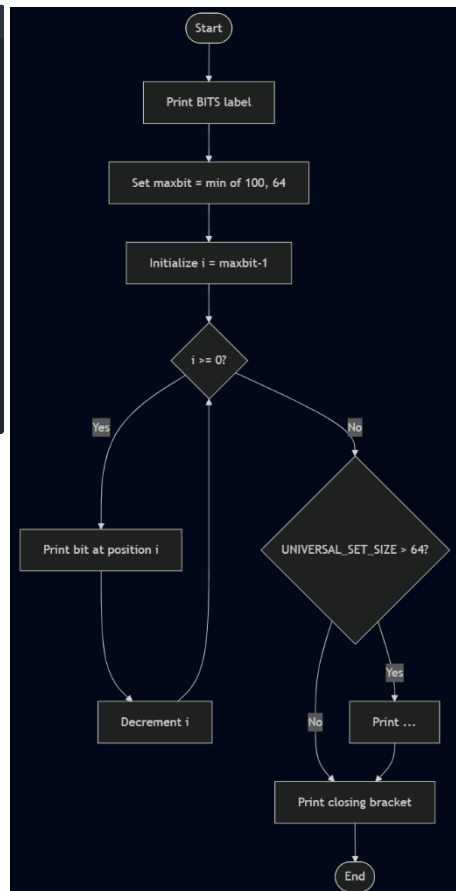
```
set_calculator.c C#  
  
double calculate_median(const Set* s) {  
    int arr[UNIVERSAL_SET_SIZE], n;  
    bits_to_set(s->bits, arr, &n); // ฟังก์ชันนี้คืนค่าเรียงลำดับอยู่แล้ว  
    if (n == 0) return 0.0;  
  
    if (n % 2 != 0) {  
        return (double)arr[n / 2];  
    } else {  
        return (double)(arr[(n - 1) / 2] + arr[n / 2]) / 2.0;  
    }  
}
```



2.3.4] กลุ่มที่ 4 ฟังก์ชันแสดงผล (Display):

2.3.4.1) ฟังก์ชัน print_set()

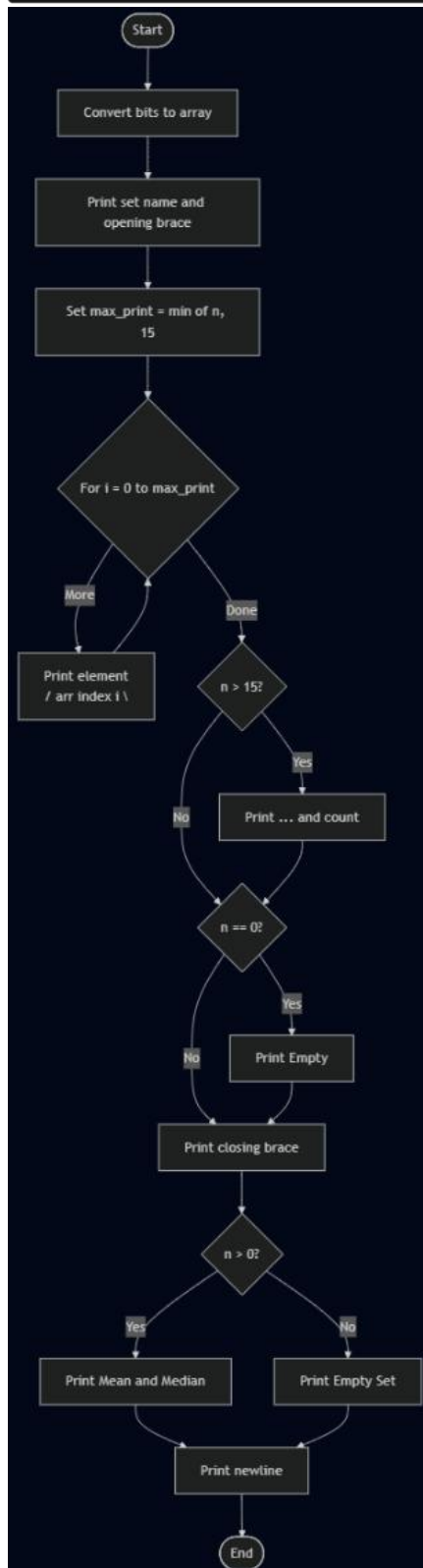
```
C set_calculator.c  
  
void print_set(const Set* s) {  
    int arr[UNIVERSAL_SET_SIZE], n = 0;  
    bits_to_set(s->bits, arr, &n);  
  
    printf(" %-12s = { ", s->name);  
  
    int max_print = n < 15 ? n : 15;  
    for (int i = 0; i < max_print; ++i)  
        printf(i == max_print - 1 ? "%d" : "%d, ", arr[i]);  
  
    if (n > 15) printf(" ...(%d)", n);  
    else if (n == 0) printf("Empty");  
  
    printf(" ]");  
  
    // เปรียบเทียบ Mean และ Median หน่อย  
    if (n > 0) {  
        printf(" [Mean: %.2f, Median: %.2f]", calculate_mean(s), calculate_median(s));  
    } else {  
        printf(" [Empty Set]");  
    }  
  
    printf("\n");  
}
```





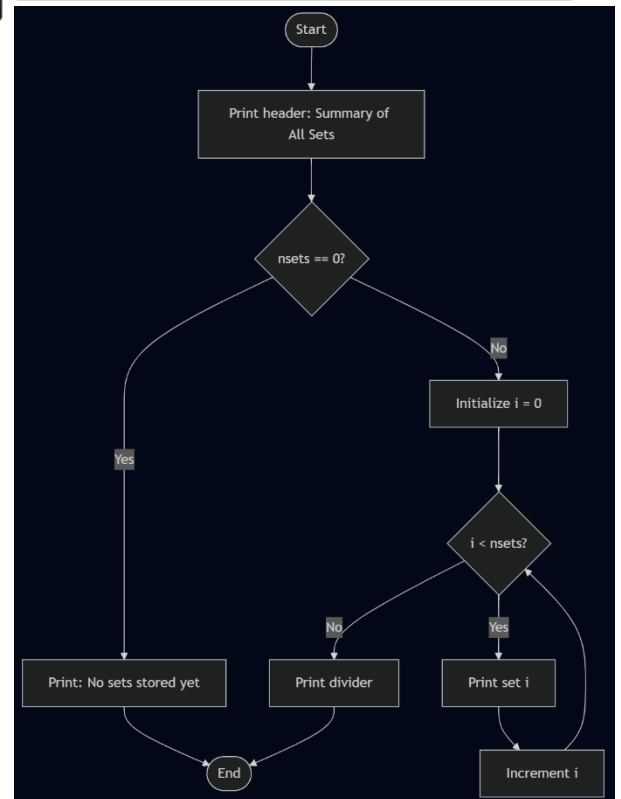
2.3.4.2) ฟังก์ชัน print_bits()

```
set_calculator.c C++
void print_bits(const unsigned char* bits) {
    printf(" [BITS: ");
    int maxbit = UNIVERSAL_SET_SIZE < 64 ? UNIVERSAL_SET_SIZE : 64;
    for (int i = maxbit-1; i >= 0; --i)
        printf("%d", get_bit(bits, i));
    if (UNIVERSAL_SET_SIZE > 64) printf("...");
    printf("]");
}
```



2.3.4.3) ฟังก์ชัน show_all_sets()

```
set_calculator.c SCSS
void show_all_sets(const Set* sets, int nsets) {
    ui_print_header("Summary of All Sets (with Stats)");
    if (nsets == 0) { printf(" (No sets stored yet)\n"); return; }
    for (int i = 0; i < nsets; ++i) print_set(&sets[i]);
    ui_print_divider();
}
```

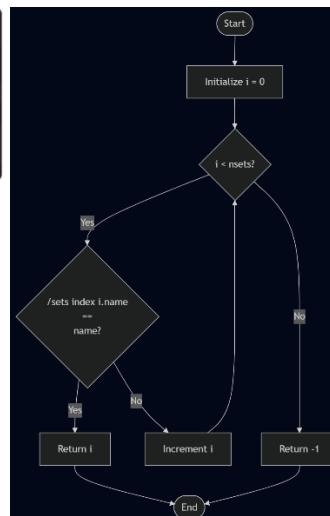




2.3.51 กลุ่มที่ 5 ฟังก์ชันจัดการข้อมูลเซต (Set Management):

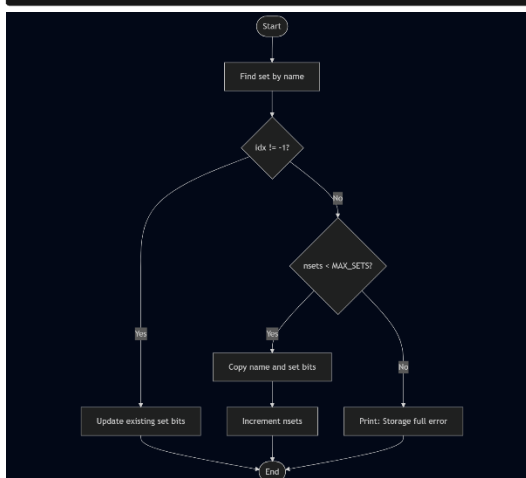
2.3.5.1) ฟังก์ชัน find_set()

```
set_calculator.c C++
int find_set(const Set* sets, int nsets, const char* name) {
    for (int i = 0; i < nsets; ++i)
        if (strcmp(sets[i].name, name) == 0) return i;
    return -1;
}
```



2.3.5.2) ฟังก์ชัน add_or_update_set()

```
set_calculator.c C++
void add_or_update_set(Set* sets, int* nsets, const char* name, int* arr, int n) {
    int idx = find_set(sets, *nsets, name);
    if (idx != -1) {
        set_to_bits(sets[idx].bits, arr, n);
    } else if (*nsets < MAX_SETS) {
        strcpy(sets[*nsets].name, name);
        set_to_bits(sets[*nsets].bits, arr, n);
        (*nsets)++;
    } else {
        printf(" [!] Error: Storage full. Cannot add '%s'\n", name);
    }
}
```



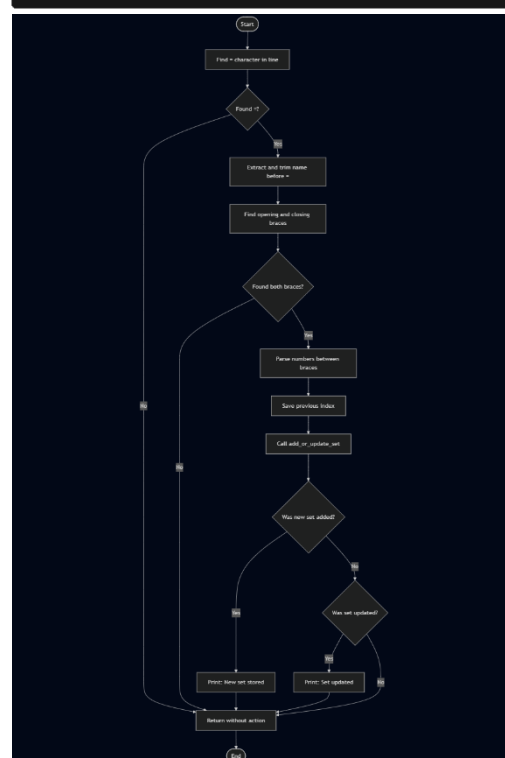
2.3.5.3) ฟังก์ชัน parse_and_add_set()

```
set_calculator.c C++
void parse_and_add_set(char* line, Set* sets, int* nsets) {
    char name[MAX_NAME];
    int arr[UNIVERSAL_SET_SIZE], n = 0, x;
    char *eq = strchr(line, '=');
    if (!eq) return;
    strncpy(name, line, eq - line);
    name[eq - line] = '\0';
    trim(name);

    char* b = strchr(line, '{');
    char* e = strchr(line, '}');
    if (!b || !e) return;
    b++;
    while (sscanf(b, "%d", &x) == 1) {
        arr[n++] = x;
        b = strchr(b, ',');
        if (!b || b > e) break;
        b++;
    }

    int prev_idx = find_set(sets, *nsets, name);
    add_or_update_set(sets, nsets, name, arr, n);

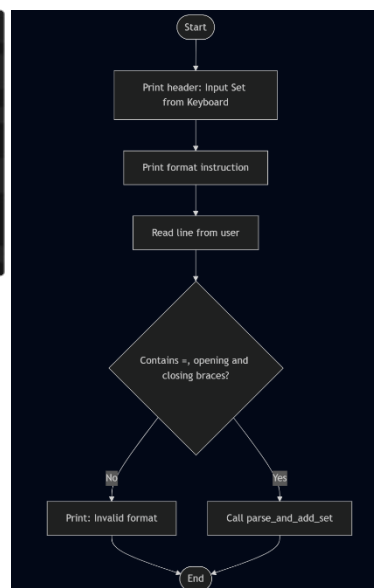
    if (prev_idx == -1 && find_set(sets, *nsets, name) != -1) {
        printf(" [+] New set stored: %s\n", name);
    } else if (prev_idx != -1) {
        printf(" [*] Set updated: %s\n", name);
    }
}
```





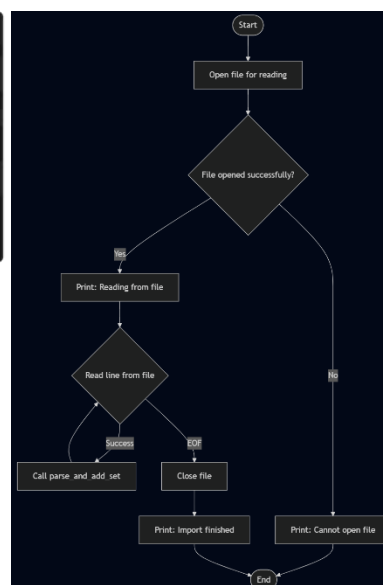
2.3.5.4) ฟังก์ชัน input_set_keyboard()

```
set_calculator.c C++
void input_set_keyboard(Set* sets, int* nsets) {
    ui_print_header("Input Set from Keyboard");
    char line[20000];
    printf(" Format: Name = {1, 2, 3}\n");
    printf(" Enter set > ");
    fgets(line, sizeof(line), stdin);
    if (!strcmp(line, '=') || !strcmp(line, '{') || !strcmp(line, '}')) {
        printf(" [!] Invalid format.\n");
        return;
    }
    parse_and_add_set(line, sets, nsets);
}
```



2.3.5.5) ฟังก์ชัน input_set_file()

```
set_calculator.c C++
void input_set_file(const char* fname, Set* sets, int* nsets) {
    FILE* f = fopen(fname, "r");
    if (!f) { printf(" [!] Cannot open file '%s'\n", fname); return; }
    char line[20000];
    printf(" Reading from '%s'...\n", fname);
    while (fgets(line, sizeof(line), f)) {
        parse_and_add_set(line, sets, nsets);
    }
    fclose(f);
    printf(" [Done] Import finished.\n");
}
```





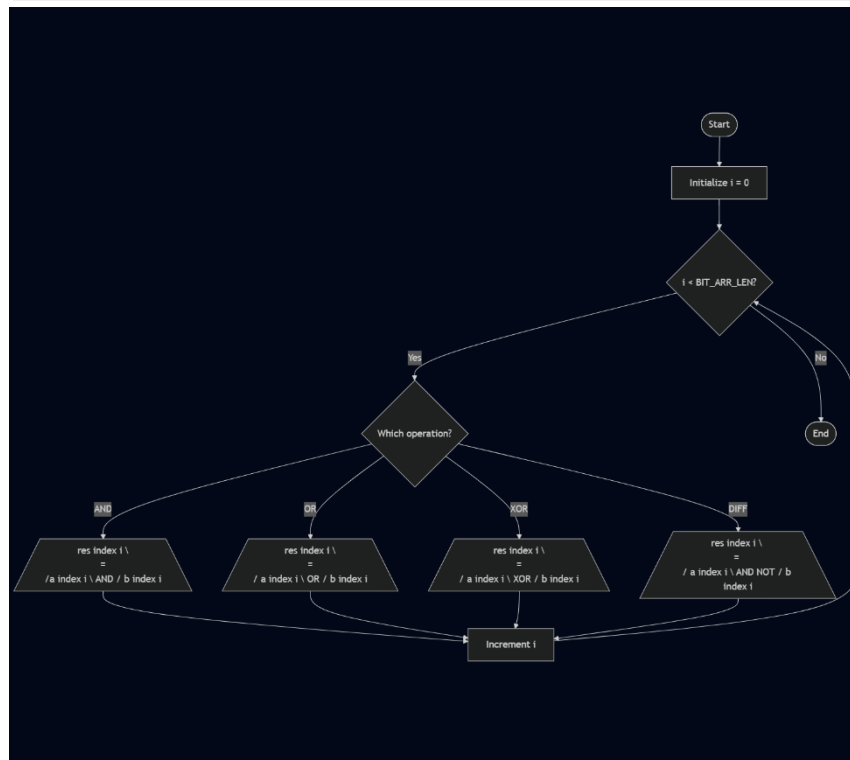
2.3.61 กลุ่มที่ 6 ฟังก์ชันคำนวณ (Operations):

2.3.6.1) ฟังก์ชัน bitwise_operation()

set_calculator.c

Rust

```
void bitwise_operation(char op, const Set* a, const Set* b, unsigned char* res) {
    for (int i = 0; i < BIT_ARR_LEN; ++i) {
        if (op == '&') res[i] = a->bits[i] & b->bits[i];
        else if (op == '|') res[i] = a->bits[i] | b->bits[i];
        else if (op == '^') res[i] = a->bits[i] ^ b->bits[i];
        else if (op == '-') res[i] = a->bits[i] & ~b->bits[i];
    }
}
```

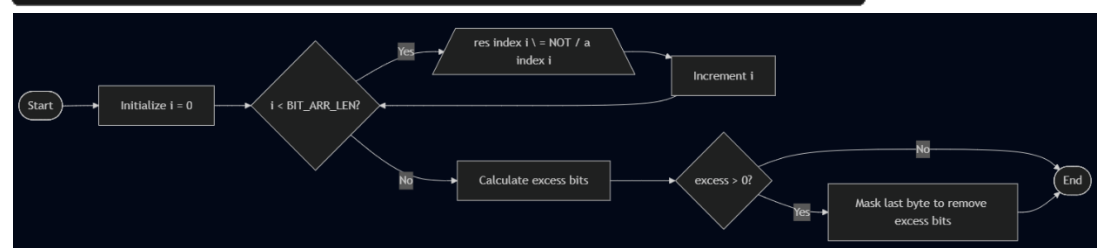


2.3.6.2) ฟังก์ชัน bitwise_not()

set_calculator.c

Rust

```
void bitwise_operation(char op, const Set* a, const Set* b, unsigned char* res) {
    for (int i = 0; i < BIT_ARR_LEN; ++i) {
        if (op == '&') res[i] = a->bits[i] & b->bits[i];
        else if (op == '|') res[i] = a->bits[i] | b->bits[i];
        else if (op == '^') res[i] = a->bits[i] ^ b->bits[i];
        else if (op == '-') res[i] = a->bits[i] & ~b->bits[i];
    }
}
```





2.3.6.3) ฟังก์ชัน do_bitwise_op()

```
set_calculator.c C++

void do_bitwise_op(Set* sets, int* nsets) {
    ui_clear_screen();
    show_all_sets(sets, *nsets);

    if (*nsets == 0) return;

    char a[MAX_NAME], b[MAX_NAME], op;
    ui_print_divider();
    printf(" [Calculation Mode]\n");

    printf(" 1st Set Name > ");
    fgets(a, sizeof(a), stdin); a[strcspn(a, "\n")] = 0; trim(a);
    int ia = find_set(sets, *nsets, a);
    if (ia == -1) { printf(" [!] Set not found.\n"); return; }

    printf(" Operation (&, |, ^, -, ~) > ");
    scanf(" %c", &op); getchar();

    unsigned char res[BIT_ARR_LEN];
    Set temp;
    strcpy(temp.name, "RESULT");

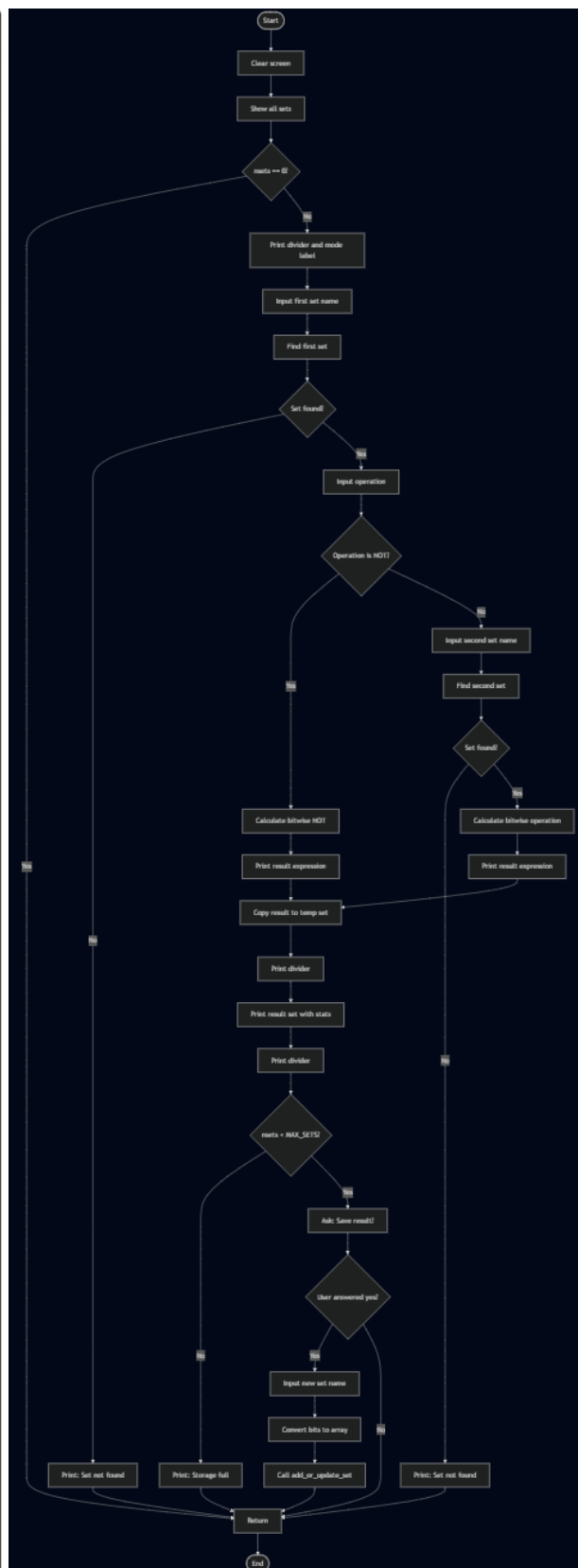
    if (op == '~') {
        bitwise_not(&sets[ia], res);
        printf("\n Result: ~%s\n", sets[ia].name);
    } else {
        printf(" 2nd Set Name > ");
        fgets(b, sizeof(b), stdin); b[strcspn(b, "\n")] = 0; trim(b);
        int ib = find_set(sets, *nsets, b);
        if (ib == -1) { printf(" [!] Set not found.\n"); return; }

        bitwise_operation(op, &sets[ia], &sets[ib], res);
        printf("\n Result: %s %c %s\n", sets[ia].name, op, sets[ib].name);
    }

    memcpy(temp.bits, res, BIT_ARR_LEN);
    ui_print_divider();
    print_set(&temp); // ผลลัพธ์ที่จะโชว์ Mean/Median ด้วย
    ui_print_divider();

    // Logic การบันทึก
    if (*nsets < MAX_SETS) {
        char save_choice;
        printf(" Save this result? (y/n) > ");
        scanf(" %c", &save_choice); getchar();
        if (save_choice == 'y' || save_choice == 'Y') {
            char new_name[MAX_NAME];
            printf(" New Set Name > ");
            fgets(new_name, sizeof(new_name), stdin);
            new_name[strcspn(new_name, "\n")] = 0; trim(new_name);

            int arr[UNIVERSAL_SET_SIZE], n;
            bits_to_set(res, arr, &n);
            add_or_update_set(sets, nsets, new_name, arr, n);
        } else {
            printf(" [!] Storage full, cannot save.\n");
        }
    }
}
```





2.3.6.4 ฟังก์ชัน save_results_file()

```
void save_results_file(const Set* sets, int nsets) {
    ui_print_header("Save to File");
    char fname[128];
    printf(" Enter filename > ");
    fgets(fname, sizeof(fname), stdin); fname[strlen(fname)] = 0;

    FILE* f = fopen(fname, "w");
    if (!f) { printf(" [!] Could not open file.\n"); return; }

    for (int i = 0; i < nsets; ++i) {
        int arr[UNIVERSAL_SET_SIZE], n;
        bits_to_set(sets[i].bits, arr, &n);
        fprintf(f, "%s = {", sets[i].name);
        int max_print = n < 20 ? n : 20;
        for (int k = 0; k < max_print; ++k)
            fprintf(f, k == max_print-1 ? "%d" : "%d,", arr[k]);
        if (n > 20) fprintf(f, ", ... (%d items)", n);

        fprintf(f, "] [bits: ");
        int maxbit = UNIVERSAL_SET_SIZE < 64 ? UNIVERSAL_SET_SIZE : 64;
        for (int m = maxbit-1; m >= 0; --m)
            fprintf(f, "%d", get_bit(sets[i].bits, m));
        if (UNIVERSAL_SET_SIZE > 64) fprintf(f, " ...");
        fprintf(f, "]\n");
    }
}
```





บทที่ 3

วิธีการใช้งานและการทดสอบการทำงานของโปรแกรม

3.1 การติดตั้งและคอมไพล์โปรแกรม (Installation and Compilation)

เนื่องจากโปรแกรมถูกพัฒนาด้วยภาษา C ผู้ใช้งานจำเป็นต้องทำการคอมไพล์ซอร์สโค้ดให้เป็นไฟล์ที่สามารถรันได้ (Executable File) ก่อนใช้งาน โดยมีขั้นตอนดังนี้:

3.1.1] เปิดโปรแกรม Terminal (บน macOS/Linux) หรือ Command Prompt (บน Windows)

3.1.2] เปลี่ยนไดเรกทอรี (Directory) ไปยังโฟลเดอร์ที่เก็บไฟล์ซอร์สโค้ด

3.1.3] พิมพ์คำสั่งสำหรับคอมไพล์ดังนี้: `gcc main.c set_calculator.c -o set_calc`

3.1.4] หากไม่มีข้อผิดพลาด ระบบจะสร้างไฟล์ `set_calc.exe` (Windows) หรือ `set_calc` (macOS/Linux) ขึ้นมา พร้อมสำหรับการใช้งาน

3.2 คู่มือการใช้งานโปรแกรม (User Manual)

เมื่อเริ่มรันโปรแกรม จะปรากฏเมนูหลัก (Main Menu) จำนวน 6 ตัวเลือก โดยมีวิธีการใช้งานแต่ละส่วนดังนี้:

```
+-----+
|          SET BITWISE CALCULATOR PRO          |
+-----+
|  1. Input set (Keyboard)                       |
|  2. Input set (File)                           |
|  3. Summary all sets (Mean & Median)           |
|  4. Calculate (Bitwise Ops)                    |
|  5. Save results to file                       |
|  6. Exit                                       |
+-----+
Select menu [1-6] > |
```

ภาพที่ 3.1 หน้าหลักของโปรแกรม

3.2.1] การป้อนข้อมูลผ่านแป้นพิมพ์ (Menu 1: Input set Keyboard)

3.2.1.1) เลือกเมนูที่ 1 แล้วกด Enter

3.2.1.2) พิมพ์นิยามเซตตามรูปแบบ เช่น `'A = {1, 2, 4, 7}'`

3.2.1.3) กด Enter เพื่อยืนยัน โปรแกรมจะแจ้งสถานะการบันทึกข้อมูล

3.2.2] การนำเข้าข้อมูลจากไฟล์ (Menu 2: Input set File)

3.2.2.1) เตรียมไฟล์ข้อความ (เช่น `'sets.txt'`) ที่มีข้อมูลเซตบรรทัดละ 1 เซต

3.2.2.2) เลือกเมนูที่ 2 แล้วกด Enter



3.2.2.3) ระบุชื่อไฟล์ที่ต้องการนำเข้า (เช่น 'sets.txt') แล้วกด Enter โปรแกรมจะอ่านและบันทึกข้อมูลทั้งหมดเข้าสู่ระบบทันที

3.2.3] การแสดงผลสรุปและสถิติ (Menu 3: Show all sets)

3.2.3.1) เลือกเมนูที่ 3 แล้วกด Enter

3.2.3.2) หน้าจอจะแสดงรายการเซตทั้งหมด พร้อมรายละเอียดสมาชิกในเซต

3.2.3.3) โปรแกรมจะคำนวณและแสดงค่า Mean (ค่าเฉลี่ย) และ Median (ค่ามัธยฐาน) ของแต่ละเซตกำกับไว้ด้านท้าย

3.2.4] การคำนวณทางเซต (Menu 4: Calculate Bitwise Ops)

3.2.4.1) เลือกเมนูที่ 4 แล้วกด Enter

3.2.4.2) ระบุชื่อ เซตที่ 1 (Set A)

3.2.4.3) ระบุ เครื่องหมายดำเนินการ ('&' ยูเนียน, '|' อินเตอร์เซกชัน, '^' ผลต่างสมมาตร, '-' ผลต่าง, '~' คอมพลีเมนต์)

3.2.4.4) ระบุชื่อ เซตที่ 2 (Set B) (ยกเว้นกรณีใช้ '~' ไม่ต้องระบุเซตที่ 2)

3.2.4.5) โปรแกรมจะแสดงผลลัพธ์การคำนวณ และถามว่าต้องการบันทึกผลลัพธ์เป็นเซตใหม่หรือไม่ ('Save this result?')

3.2.4.6) หากต้องการบันทึก ให้ตอบ 'y' และตั้งชื่อเซตใหม่

3.2.5] การบันทึกผลลัพธ์ลงไฟล์ (Menu 5: Save results to file)

3.2.5.1) เลือกเมนูที่ 5 แล้วกด Enter

3.2.5.2) ระบุชื่อไฟล์ปลายทางที่ต้องการบันทึก (เช่น 'output.txt')

3.2.5.3) โปรแกรมจะทำการเขียนข้อมูลเซตปัจจุบันทั้งหมดลงสู่ไฟล์ดังกล่าว

3.2.6] การออกจากโปรแกรม (Menu 6: Exit)

3.2.6.1) เลือกเมนูที่ 6 เพื่อปิดการทำงานของโปรแกรม

3.2.6.2) ระบุชื่อไฟล์ปลายทางที่ต้องการบันทึก (เช่น 'output.txt')

3.2.6.3) โปรแกรมจะทำการเขียนข้อมูลเซตปัจจุบันทั้งหมดลงสู่ไฟล์ดังกล่าว



3.3 การทดสอบการทำงานของโปรแกรม (System Testing)

เพื่อให้มั่นใจว่าโปรแกรมทำงานถูกต้อง แม่นยำ และสอดคล้องกับทฤษฎีเซต จึงได้กำหนดกรณีทดสอบ (Test Cases) โดยใช้ชุดข้อมูลตัวอย่างดังนี้:

ชุดข้อมูลทดสอบ: เอกภพสัมพัทธ์ $U = \{0, 1, \dots, 31\}$ (ตามการตั้งค่า Config ซึ่งสามารถแก้ไขได้)

$$A = \{1, 2, 4, 7\}$$

$$B = \{0, 1, 2, 3, 5\}$$

3.3.1 การทดสอบการนำเข้าและวิเคราะห์สถิติ (Import & Statistics Test)

ขั้นตอนการทดสอบ	ข้อมูลนำเข้า (Input)	ผลลัพธ์ที่คาดหวัง (Expected Output)	ผลการทดสอบ
1. นำเข้าไฟล์	ไฟล์ sets.txt	ระบบแจ้ง "Import finished"	ผ่าน
2. ตรวจสอบค่าสถิติ	เลือกเมนู 3	Set A: {1, 2, 4, 7} Mean: 3.50, Median: 3.00 Set B: {0, 1, 2, 3, 5} Mean: 2.20, Median: 2.00	ผ่าน

```
=====
Import from File
=====
Filename > sets.txt
Reading from 'sets.txt'...
[+] New set stored: A
[+] New set stored: B
[Done] Import finished.
[Press Enter to continue...]
```

```
=====
Summary of All Sets (with Stats)
=====
A          = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
B          = { 0, 1, 2, 3, 5 } [Mean: 2.20, Median: 2.00]
-----
[Press Enter to continue...]
```

ภาพที่ 3.2 ผลการทดสอบการนำเข้าไฟล์ แสดงผล และค่าสถิติ (Show all sets)



3.3.2] การทดสอบการคำนวณ (Calculation Test)

การดำเนินการ	กรณีทดสอบ	การคำนวณทางทฤษฎี	ผลลัพธ์จากโปรแกรม	ผลการทดสอบ
Intersection (\$\cap\$)	A & B	{1, 2}	Result: { 1, 2 }	ผ่าน
Union (\$\cup\$)	A B	{0, 1, 2, 3, 4, 5, 7}	Result: { 0, 1, 2, 3, 4, 5, 7 }	ผ่าน
Symmetric Difference (\$\oplus\$)	A ^ B	{0, 3, 4, 5, 7}	Result: { 0, 3, 4, 5, 7 }	ผ่าน
Difference (\$-\$)	A - B	{4, 7}	Result: { 4, 7 }	ผ่าน
Complement (\$'\$)	A ~	{0, 3, 5, 6, 8, ..., 31}	Result: {0, 3, 5, 6, 8, ..., (31)}	ผ่าน

```
=====
Summary of All Sets (with Stats)
=====
A          = { 1, 2, 4, 7 } [Mean: 3.50, Median: 3.00]
B          = { 0, 1, 2, 3, 5 } [Mean: 2.20, Median: 2.00]
-----

[Calculation Mode]
1st Set Name > A
Operation (&, |, ^, -, ~) > &
2nd Set Name > B

Result: A & B
-----
RESULT      = { 1, 2 } [Mean: 1.50, Median: 1.50]
-----
Save this result? (y/n) > |
```

ภาพที่ 3.3 ผลการทดสอบการนำเข้าไฟล์ แสดงผล และค่าสถิติ (Show all sets)



บทที่ 4

สรุปผลการทำ Term Project

จากการดำเนินโครงการพัฒนาโปรแกรม Set Bitwise Calculator เพื่อการจัดการและวิเคราะห์เซต ด้วยเทคนิคการดำเนินการระดับบิต (Bitwise Operations) คณะผู้จัดทำสามารถสรุปผลการดำเนินงาน ปัญหาและอุปสรรค รวมถึงข้อเสนอแนะได้ดังนี้

4.1 สรุปผลการดำเนินงาน (Summary of Implementation)

การพัฒนาโปรแกรมในครั้งนี้บรรลุวัตถุประสงค์ที่ตั้งไว้ทุกประการ โดยสามารถสร้างซอฟต์แวร์ต้นแบบ ที่ทำงานบนระบบปฏิบัติการผ่านหน้าจอ Console (CLI) ซึ่งมีจุดเด่นสำคัญดังนี้:

4.1.1] ประสิทธิภาพการคำนวณ: โปรแกรมสามารถดำเนินการทางเซตพื้นฐาน ($\cap$, $\cup$, $-$, $\sim$, $\oplus$) ได้อย่างถูกต้องและรวดเร็ว โดยใช้หลักการ Bitwise Mapping ซึ่งช่วยลดความซับซ้อนของอัลกอริทึมและประหยัดพื้นที่หน่วยความจำเมื่อเทียบกับการใช้อาร์เรย์แบบดั้งเดิม

4.1.2] การวิเคราะห์ข้อมูล: โปรแกรมสามารถคำนวณและแสดงค่าสถิติเชิงพรรณนา ได้แก่ ค่าเฉลี่ยเลขคณิต (Mean) และค่ามัธยฐาน (Median) ของสมาชิกในเซตได้ทันที ช่วยให้ผู้ใช้งานได้รับข้อมูลเชิงลึกประกอบการตัดสินใจ

4.1.3] ความยืดหยุ่นในการใช้งาน: ระบบรองรับการนำเข้าข้อมูลทั้งจากแป้นพิมพ์และไฟล์ข้อความ (.txt) รวมถึงสามารถบันทึกผลลัพธ์การคำนวณกลับลงสู่ไฟล์ได้ ทำให้การจัดการข้อมูลจำนวนมาก เป็นไปอย่างสะดวกและเป็นระบบ

4.1.4] โครงสร้างโปรแกรม: การออกแบบโค้ดแบบแยกส่วน (Modular Design) โดยแยกไฟล์ส่วนหัว (.h), ไฟล์ฟังก์ชัน (.c) และไฟล์หลัก (main.c) ออกจากกัน ช่วยให้โค้ดมีความเป็นระเบียบง่ายต่อการตรวจสอบและแก้ไข

4.2 ปัญหาและอุปสรรค (Problems and Limitations)

ในระหว่างการพัฒนาและการทดสอบระบบ พบปัญหาและข้อจำกัดบางประการ ดังนี้:

4.2.1] ขอบเขตของเอกภพสัมพัทธ์ (Universal Set Limitation): ขนาดของเอกภพสัมพัทธ์ ถูกกำหนดตายตัว (Static Allocation) ผ่านค่าคงที่ UNIVERSAL_SET_SIZE ในขั้นตอนการคอมไพล์ (Compile-time) หากผู้ใช้ต้องการใช้งานกับตัวเลขที่มีค่ามากกว่าที่กำหนดไว้ จำเป็นต้องแก้ไขโค้ดและคอมไพล์ใหม่ ไม่สามารถปรับเปลี่ยนได้ขณะรันโปรแกรม (Runtime)



4.2.2] รูปแบบการป้อนข้อมูล (Input Formatting): การนำเข้าข้อมูลเซตผ่านการพิมพ์หรือไฟล์ จำเป็นต้องรักษารูปแบบไวยากรณ์ (Syntax) อย่างเคร่งครัด (เช่น $A = \{1, 2\}$) หากผู้ใช้ป้อนรูปแบบผิดเพี้ยนไปเพียงเล็กน้อย อาจส่งผลให้การอ่านค่าผิดพลาดหรือไม่บันทึกข้อมูล

4.2.3] การรองรับสมการซับซ้อน: ปัจจุบันโปรแกรมรองรับการคำนวณทีละขั้นตอน (Step-by-step) ระหว่าง 2 เซตเท่านั้น ยังไม่รองรับการป้อนสมการยาวๆ ที่มีวงเล็บซ้อนกัน เช่น $(A \cup B) \cap C$ ในครั้งเดียว

4.3 ข้อเสนอแนะ (Suggestions for Future Development)

เพื่อเพิ่มประสิทธิภาพและขีดความสามารถของโปรแกรมในอนาคต คณะผู้จัดทำมีข้อเสนอแนะดังนี้:

4.3.1] การพัฒนาการแปลสมการ (Expression Parser): ควรเพิ่มอัลกอริทึมสำหรับแปลงนิพจน์ทางคณิตศาสตร์ (เช่น Shunting-yard algorithm) เพื่อให้รองรับการคำนวณสมการเซตที่ซับซ้อนและมีวงเล็บได้ในคำสั่งเดียว

4.3.2] การพัฒนาส่วนติดต่อกราฟิก (GUI Development): พัฒนาส่วนติดต่อผู้ใช้ในรูปแบบกราฟิก (Graphical User Interface) เพื่อให้มีความสวยงาม ทันสมัย และใช้งานได้ง่ายกว่ารูปแบบ Console

4.3.3] การเพิ่มฟังก์ชันทางสถิติ: เพิ่มการคำนวณค่าทางสถิติอื่นๆ เช่น ส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation) หรือความแปรปรวน (Variance) เพื่อให้เป็นเครื่องมือวิเคราะห์ข้อมูลที่สมบูรณ์ยิ่งขึ้น